



Today's agenda

- ↳ Subarray sum equals k
- ↳ Same differences
- ↳ Subarray with equal 0 and 1.
- ↳ Subarray with equal 0, 1 and 2. ~~Hard~~

→ Pramp.Com



AlgoPrep



Q) Subarray sum equals K

↳ Given $arr[n]$ and an integer K , return total number of Continuous subarrays whose sum equals to K .

Ex: $arr[3] = \{1, 1, 1\}$ $K=2$ $ans=2$

// Idea 1

↳ Check all subarrays and Count if $Sum == K$.

T.C: $O(N^2)$

S.C: $O(N)$

// Idea 2 → find ep and Count valid sp in $O(1)$.

$K=4$

	0	1	2	3	4	5	6	7	8	9	10	ep
$arr[11]$	2	3	-1	4	-5	1	6	-4	3	-1	4	
$Psum[11]$	2	5	4	8	3	4	10	6	9	8	12	

$$Sum(sp, ep) == K$$

$ep=4$

$$Psum[ep] - Psum[sp-1] == K$$

$$3 - Psum[sp-1] = 4$$

$$-1 = Psum[sp-1]$$

K=4

arr[11] = 2 3 -1 4 -5 1 6 -4 3 -1 4
Psum[11] = 2 5 4 8 3 4 10 6 9 8 12

$$Psum[ep] - Psum[sp-1] = K$$

$$Psum[ep] - K = Psum[sp-1]$$

$$8 - 4 = Psum[sp-1] = 4$$

K=4

arr[11] = 2 3 -1 4 -5 1 6 -4 3 -1 4
Psum[11] = 2 5 4 8 3 4 10 6 9 8 12

ep

arr[ep]

$$Psum[ep] - K = Psum[sp-1]$$

valid sp

map

0	2	-2	0
1	5	1	0
2	4	0	1
3	8	4	1
4	3	-1	0
5	4	0	1
6	10	6	0

0 - 1	3 - 1
2 - 1	10 - 1
5 - 1	
4 - 2	
8 - 1	



//Pseudo code

```
int subarraySum (int arr[], int k) {  
    HashMap<Integer, Integer> map;  
    map.put (0, 1);  
    int ans = 0;  
  
    int[] Psum = func1 (arr);  
  
    for (int ep = 0; ep < N; ep++) {  
        int diff = Psum[ep] - k;  
        ans = ans + map.getOrDefault (diff, 0);  
  
        map.put (Psum[ep], map.getOrDefault (Psum[ep], 0) + 1);  
    }  
  
    return ans;  
}
```

T.C: $O(N)$

S.C: $O(N)$



```
int subarraySum (int arr[], int k) {
```

```
    HashMap<Integer, Integer> map;
```

```
    map.put (0,1);
```

```
    int ans = 0;
```

```
    int[] psum = func1(arr);
```

```
    for (int ep=0; ep<N; ep++) {
```

```
        int diff = psum[ep] - k;
```

```
        ans = ans + map.getOrDefault(diff, 0);
```

```
        map.put (psum[ep], map.getOrDefault(psum[ep], 0) + 1);
```

```
    }
```

```
    return ans;
```

```
}
```

arr[] = 0 2 3 -1 4 -5 1 6 -4 3 -1 4
 psum[] = 0 2 5 4 8 3 4 10 6 9 8 12

Annotations:
 -1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 (above arr)
 ep (above 2, 3, -1)
 ep (above 4)
 ep (above 8)

ep arr[ep] psum[ep] - k = psum[sp-1] valid sp

2 4 0

1

map

0-1

2-1

5-1



AlgoPrep



Q) Same differences

↳ Given $a[0..N]$, Count the number of pairs of indices (i, j) such that $i < j$ and $a_j - a_i = j - i$

$$a[0..5] = \{ \overset{0}{3} \ \overset{1}{5} \ \overset{2}{1} \ \overset{3}{4} \ \overset{4}{6} \ \overset{5}{6} \}$$

$$a_j - a_i = j - i \Rightarrow a_j - j = a_i - i$$

// Idea 1 → for every j , iterate and find valid i .

$$a[0..5] = \{ \overset{0}{3} \ \overset{1}{5} \ \overset{2}{1} \ \overset{3}{4} \ \overset{4}{6} \ \overset{5}{6} \}$$

j	$a_j - j$	valid i
0	3	0
1	4	0
2	-1	0
3	1	0
4	2	0
5	1	1

ans = 1

T.C: $O(N^2)$

S.C: $O(1)$



1/idea2

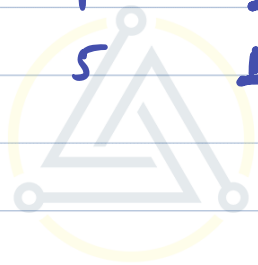
j

arr[6] = { 3 5 1 4 6 6 }

i	a _j - j	valid i
0	3	0
1	4	0
2	-1	0
3	1	0
4	2	0
5	1	1

ans = 1

3 - 1
4 - 1
-1 - 1
1 - 2
2 - 1



AlgoPrep



//Pseudo Code

T.C: $O(N)$

S.C: $O(N)$

```
int SameDifferences (int arr[N]) {  
    HashMap < Integer, Integer > map;  
    int ans = 0;  
    for (int j=0; j<N; j++) {  
        int diff = arr[j] - j;  
        ans = ans + map.getOrDefault(diff, 0);  
        map.put (diff, map.getOrDefault(diff, 0) + 1);  
    }  
    return ans;  
}
```

Break till 9:12 PM



Q) Count Subarrays with equal number of 1's and 0's.

↳ Given $arr[n]$ with 0's and 1's only. find the number of Subarrays with equal number of 0's and 1's.

Ex: $arr[4] = \{1, 0, 0, 1\} \rightarrow ans = 3$

Idea 1

↳ Replace 0 with -1 and find number of Subarrays with $sum = 0$.

$arr[4] = \{1, 0, 0, 1\}$

$arr[4] = \{1, -1, -1, 1\}$

T.C: $O(n)$

S.C: $O(n)$

Idea 2

$arr[8] = \{0, 1, 0, 0, 1, 0, 0, 1\}$

$PC0[8] = \{1, 1, 2, 3, 3, 4, 5, 5\}$

$PC1[8] = \{0, 1, 1, 1, 2, 2, 2, 3\}$

$Count_0(sp, ep) == Count_1(sp, ep)$

$PC0[ep] - PC0[sp-1] == PC1[ep] - PC1[sp-1]$

$PC0[ep] - PC1[ep] == PC0[sp-1] - PC1[sp-1]$



$$P_{co}[ep] - P_{cl}[ep] == P_{co}[sp-1] - P_{cl}[sp-1]$$

$arr[8]: \{ 0 \ 1 \ 0 \ 0 \ 1 \ 0 \ 0 \ 1 \}$

$P_{co}[8]: \ 1 \ 1 \ 2 \ 3 \ 3 \ 4 \ 5 \ 5$

$P_{cl}[8]: \ 0 \ 1 \ 1 \ 1 \ 2 \ 2 \ 2 \ 3$

ep	$P_{co}[ep] - P_{cl}[ep]$	valid sp-1
0	$1 - 0 = 1$	0
1	$1 - 1 = 0$	1
2	$2 - 1 = 1$	1
3	$3 - 1 = 2$	0
4	$3 - 2 = 1$	2
5	$4 - 2 = 2$	1
6	$5 - 2 = 3$	0
7	$5 - 3 = 2$	2

↳ if iterate for every ep, T.C : $O(n^2)$

Wing Hashmap



ep

arr[8]: { 0 1 0 0 1 0 0 1 }

Pco[8]: 1 1 2 3 3 4 5 5

Pc1[8]: 0 1 1 1 2 2 2 3

map

ep	Pco[ep] - Pc1[ep]	valid sp-1
0	1	0
1	0	1
2	1	1
3	2	0
4	1	2
5	2	1

0-12
1-123
2-12





11PSuedo Code

```
int countSubarrayOL (int arr[N]) {  
    HashMap < Integer, Integer > map;  
  
    int [] PC0 = func0(arr);  
    int [] PC1 = func1(arr);  
    int ans = 0;  
    map.put (0, 1);  
  
    for (int ep = 0; ep < N; ep++) {  
        int diff = PC0[ep] - PC1[ep];  
  
        ans = ans + map.getOrDefault(diff, 0);  
  
        map.put (diff, map.getOrDefault(diff, 0) + 1);  
    }  
  
    return ans;  
}
```

T.C: $O(N)$

S.C: $O(N)$



Q) Count Subarrays with equal number of 0's and 1's and 2's.
↳ Given $arr[n]$ with 0's, 1's and 2's. find the number of Subarrays with equal number of 0's, 1's and 2's.

Ex: $arr[7] = \{0, 1, 0, 2, 0, 1, 0\} \rightarrow ans = 2$

//idea 1 $\rightarrow$ incorrect idea

↳ Replace 2 with -1 and find the subarray with sum = 0. $\rightarrow \{-1, 0, 1\}$

$arr[3] = \{0, 0, 0\} \xrightarrow{rep} ans = 0$
 $1 + 2 + 3 = 6$

//idea 2

$arr[7] = \{0, 1, 0, 2, 0, 1, 0\}$

$pc0[7] = \{1, 1, 2, 2, 3, 3, 4\}$

$pc1[7] = \{0, 1, 1, 1, 1, 2, 2\}$

$pc2[7] = \{0, 0, 0, 1, 1, 1, 1\}$



arr[7]: { 0⁰ 1¹ 0² 2² 0⁴ 1⁵ 0⁶ }

pc0[7]: 1 1 2 2 3 3 4

pc1[7]: 0 1 1 1 1 2 2

pc2[7]: 0 0 0 1 1 1 1

"1@1" "0@1" "1@2" "1@1" "2@2" "1@2" "2@3"

ep	pc0[ep] - pc1[ep]	pc0[ep] - pc2[ep]	valid str
0	1 - 0 = 1	1	0
1	0	1	0
2	1	2	0
3	1	1	1
4	2	2	0
5	1	2	1
6	2	3	0



// Pseudo code

```
int countSubarray012 ( int arr[N] ) {  
    HashMap < String, Integer > map;
```

```
    int[] P0 = func0(arr);  
    int[] P1 = func1(arr);  
    int[] P2 = func2(arr);
```

T.C: $O(N)$

S.C: $O(N)$

```
    int ans = 0;  
    map.put ("0@0", 1);
```

```
    for (int ep = 0; ep < N; ep++) {  
        int diff1 = P0[ep] - P1[ep];  
        int diff2 = P0[ep] - P2[ep];  
        String diff = diff1 + "@" + diff2;
```

```
        ans = ans + map.getOrDefault(diff, 0);
```

```
        map.put(diff, map.getOrDefault(diff, 0) + 1);
```

```
    }
```

```
    return ans;
```

```
}
```



```
int countSubarray012 ( int arr[N]) {  
    HashMap<String, Integer> map;
```

```
    int[] pco = func0(arr);  
    int[] pci = func1(arr);  
    int[] pc2 = func2(arr);
```

```
    int ans = 0;  
    map.put ("0@0", 1);
```

```
    for (int ep = 0; ep < N; ep++) {  
        int diff1 = pco[ep] - pci[ep];  
        int diff2 = pci[ep] - pc2[ep];  
        String diff = diff1 + "@" + diff2;
```

```
        ans = ans + map.getOrDefault(diff, 0);
```

```
        map.put(diff, map.getOrDefault(diff, 0) + 1);
```

```
    }
```

```
    return ans;
```

```
}
```

arr[]: { 0 1 0 2 0 1 0 }

pco[]: 1 1 2 2 3 3 4

pci[]: 0 1 1 1 1 2 2

pc2[]: 0 0 0 1 1 1 1

"1@1" "0@1" "1@2" "1@1" "2@1" "1@2" "2@3"

ans = 0 + 0 + 0 + 0

+ 1 + 0 + 1

= 2

"0@0" → 1

"1@1" → 1 2

"0@1" → 1

2@3 → 1

"1@2" → 1 2

"2@2" → 1



AlgoPrep